

3.1 Overview

- **Objective:** Introduce the reader to the contents of the chapter.
- **What to write:** A short paragraph mentioning the main stages the project will go through.
- **Example:**
"This chapter explains the methodology followed to achieve the engineering objectives of the project. It will present [number] main stages: (1) **Data Collection** from reliable global sources, (2) **Data Preprocessing** and preparation, (3) **Building the Proposed Model**, and (4) **Performance Evaluation**. Each stage will be explained in detail, along with a justification of the engineering choices made."

3.2 Requirements & Specifications

- **Objective:** Define the criteria that the engineering solution must meet.
- **What to write:** A list of functional and non-functional requirements.
- **Example (Power Engineering):**

"Based on the problem analysis in Chapter One, the following requirements were identified for the proposed system:

- **Functional Requirements:** The system must detect ground faults within less than 100 milliseconds.
- **Non-Functional Requirements:** The system must operate with at least 95% efficiency under noisy conditions (SNR < 20 dB)."

3.3 Possible Scenarios

- **Objective:** Acknowledge that multiple methods may exist to solve the problem, and justify your choice of a specific method.
- **What to write:** Mention the possible options, then explain why you chose this particular option.
- **Example (Robotics):**

"For path planning in an unknown environment, several algorithms are available:

- **Option 1:** Traditional A* algorithm.
- **Option 2:** RRT algorithm (Rapidly-exploring Random Tree).
- **Option 3:** Reinforcement Learning algorithm.

The third option was chosen because previous studies [Cite Reference] have shown that reinforcement learning algorithms adapt better to dynamic and changing environments, which is an essential requirement in a university campus environment."

3.4 System Framework

- **Objective:** Present the big picture of the system.
- **What to write:** Provide an illustrative diagram (Block Diagram) explaining the system components and the flow of signals or data between them. Then explain the diagram in text.
- **Example (Robotics):**

"Figure (3.1) illustrates the general framework of the proposed robotic system. The system begins by receiving data from sensors, then passes through the data processing stage, followed by the decision-making stage using the trained model, and finally the motion execution stage."

3.5 Dataset Description

This section is entirely dedicated to describing the data. It must be detailed and precise.

Objective:

To accurately document the data source to ensure research transparency and reproducibility.

What to write:

1. **Data Type:**
2. **Source (with link if possible):**
3. **Data Description:**
4. **Data Size:**

5. Any modifications or preprocessing performed:

Detailed Examples by Specialization:

A. Power Engineering:

Example 1: Transmission Line Fault Data

"The "IEEE 39-bus Power System" database was used. It is a reliable global database available through the link [Add link].

- **Data Type:** Voltage and current signals for a 39-bus power network, recorded at a high frequency (10 kHz).
- **Source:** Institute of Electrical and Electronics Engineers (IEEE) in collaboration with the Pacific Northwest National Laboratory.
- **Data Description:** The database contains 5000 normal operating conditions and 5000 fault conditions (ground fault, phase-to-ground fault, three-phase fault). Each condition is recorded in a separate file containing voltage and current values for the three phases.
- **Data Size:** 10,000 files, total size 5.2 GB.
- **Modifications Made:**
 - Data was reshaped to fit the proposed neural network's input (a 1000x6 matrix).
 - White Gaussian Noise was added at different ratios (SNR from 10dB to 40dB) to simulate real operating conditions.
 - Data was normalized using the Min-Max technique to be in the range [0, 1]."

Example 2: Solar Power Data

"The "NREL Solar Power Data" database from the National Renewable Energy Laboratory (NREL) in the USA was used.

- **Data Type:** Time series data for solar irradiance, temperature, and power generated from solar panels.
- **Source:** The official NREL website [Add link].
- **Data Description:** Data recorded every 15 minutes for 5 years (2018-2023) from a station in Colorado.
- **Data Size:** Approximately 175,000 readings.
- **Modifications Made:**
 - Data was aggregated to hourly averages to simplify the model.
 - Outliers resulting from measurement equipment errors were removed.
 - Data was split: 70% training, 15% validation, 15% testing."

B. Robotics:

Example 1: Visual Navigation Data

"The "KITTI Vision Benchmark Suite" database was used, one of the most famous databases in the field of robotics and computer vision.

- **Data Type:** Images (RGB), depth data (LiDAR), and positioning data (GPS/IMU).
- **Source:** Karlsruhe Institute of Technology (KIT) and Toyota Technological Institute, Chicago. Available at [Add link].
- **Data Description:** The database contains video footage of driving cars on the streets of a German city, with labels for people, cars, and traffic signs. It was recorded at 10 frames per second.
- **Data Size:** 50,000 labeled images for training vision models.
- **Modifications Made:**
 - Images were cropped to remove the sky and unimportant parts.
 - Images were resized to 224x224 pixels to fit the input of a ResNet network.
 - Data augmentation techniques, such as simple rotation and horizontal flipping, were applied to increase data diversity and prevent overfitting."

Example 2: Robotic Arm Control Data

"The "Robotic Arm Control Dataset" from Columbia University was used.

- **Data Type:** Joint angles coordinates and end-effector position coordinates.
- **Source:** Robotics Laboratory, Columbia University [Add link].
- **Data Description:** 10,000 motion trajectories were recorded for a 6-DOF (Degrees of Freedom) robotic arm.
- **Modifications Made:**

- Coordinates were transformed from the Cartesian system to joint angles (Inverse Kinematics) to train the inverse model.
- Trajectories were smoothed using a Moving Average Filter."

3.6 Implementation Details

- **Objective:** Explain how the model or system was built after data preparation.
- **Example (Robotics):**

"The reinforcement learning model was built using the Stable-Baselines3 library in Python. The PPO algorithm (Proximal Policy Optimization) was used because it has proven efficient in navigation tasks. The model was trained for 1,000,000 time steps in a Gazebo simulation environment."

- **Example (Power Engineering):**

"An LSTM network with long short-term memory was built for electrical load forecasting. The network consists of:

- An Input Layer of size 24 (for the previous 24 hours of data).
 - Two hidden LSTM layers, each containing 50 cells.
 - An output Dense layer of size 1 (to predict the next hour).
- The Adam Optimizer was used with a Learning Rate = 0.001."

3.7 Evaluation Plan

- **Objective:** How will you prove that your system works correctly?
- **Example (Power Engineering):**

"The forecasting model was evaluated using two metrics:

- **Mean Absolute Error (MAE):** To measure the average difference between predicted and actual values.
- **Root Mean Square Error (RMSE):** To measure the standard deviation of the errors.

The results were compared with the traditional ARIMA model and the SVR model to demonstrate performance improvement."

3.8 Chapter Summary

- **Objective:** Summarize what has been explained and provide a transition to the next chapter.

3.1 Chapter Overview

- **Objective:** Introduce the reader to the contents of the chapter.
- **What to write:** A short paragraph mentioning the main stages the project will go through, emphasizing the philosophy of integrating layers and containers.
- **Example (Customized for the Financial Project):**
"This chapter explains the methodology followed to build an intelligent, multi-currency financial transfer system. The system's architecture is based on the integration of two fundamental concepts: **Logical Layers** and **Technical Containers**. **Layers** represent the logical separation of responsibilities (Presentation, Business Logic, Data, etc.), while **Containers** represent the independent, deployable units that execute the functions within each layer. This chapter will present: (1) The system requirements and specifications, (2) The possible architectural scenarios justifying the chosen approach, (3) The integrated system framework showing layers and their constituent containers, (4) The datasets used from reliable global sources, (5) Implementation details for the containers within each layer, and (6) The comprehensive evaluation plan for the system. Each stage will be explained in detail, justifying the engineering and financial choices made."

3.2 Requirements & Specifications

- **Objective:** Define the criteria that the engineering and financial solution must meet.
- **What to write:** A list of functional and non-functional requirements, including those specific to the layered/containerized architecture.
- **Example (Customized for the Financial Project):**
"Based on the problem analysis in Chapter One, and in coordination with the partners (the Central Bank and commercial banks), the following requirements were identified for the proposed system:

Functional Requirements:

- The system must execute a transfer between two different currencies in less than 3 seconds.
- The system must update exchange rates in real-time based on market data.
- The system must detect suspicious transactions (money laundering, fraud) with an accuracy of no less than 98%.
- The system must support at least 10 major world currencies (USD, EUR, GBP, SAR, EGP, etc.).
- The system must integrate with the Central Bank and commercial banks via unified APIs.

Non-Functional Requirements:

- **Scalability:** Each container must be capable of independent horizontal scaling.
- **Isolation (Layered/Container):** A fault in one container must not affect the performance of other containers or layers.
- **Security:** The system must comply with banking information security standards (PCI DSS) and data encryption (AES-256), with container-specific security scanning.
- **Reliability:** System uptime must be at least 99.9%.
- **Transparency & Auditability:** The system must be auditable by the Central Bank and regulatory bodies, especially the Data and Audit Log layers.
- **Privacy:** Must comply with data protection laws like GDPR."

3.3 Possible Scenarios

- **Objective:** Acknowledge that multiple methods exist to solve the problem, and justify your choice of a specific method (here, justifying the integrated Layers + Containers architecture).
- **What to write:** Mention the possible architectural options, then explain why you chose this particular option.
- **Example (Customized for the Financial Project):**

A. For System Architecture:

"For building an integrated financial system, several architectural models are possible:

- **Option 1: Monolithic Architecture:** The entire system is built as one single application.
 - *Advantages:* Simple to start with, easy to deploy initially.

- *Disadvantages:* Difficult to scale, any small change requires redeploying the whole system, an error in one part can crash the entire system.
- **Option 2: Layered Architecture (N-Tier):** The system is divided into logical layers (Presentation, Business, Data). This is a good logical design but often deployed as a single unit or a few large units.
 - *Advantages:* Clear separation of concerns, easier to understand.
 - *Disadvantages:* Layers are often still coupled in deployment; scaling still requires scaling the whole layer.
- **Option 3: Microservices / Container-Based Architecture:** The system is broken down into small, independent services (containers).
 - *Advantages:* Each service can be developed, deployed, and scaled independently. Excellent fault isolation.
 - *Disadvantages:* Can be complex to manage inter-service communication and orchestration.
- **Option 4: Integrated Layers + Containers Architecture (The Proposed Approach):** This combines the best of both worlds. The system is designed using clear **logical layers** (for conceptual clarity), but within each layer, the functionality is implemented as a set of independent **containers** (for technical flexibility).

The Proposed Approach (Option 4) was chosen for the following reasons:

1. **Conceptual Clarity (from Layers):** The layered design makes the system easy to understand, document, and explain to stakeholders like the Central Bank. It clearly defines responsibilities.
2. **Technical Flexibility (from Containers):** The containerized implementation allows independent scaling. For example, only the Fraud Detection Container can be scaled during peak times without affecting the User Container.
3. **Enhanced Security (from both):** Layers provide logical security boundaries. Containers provide physical/technical isolation. An attacker breaching the Web App Container in the Presentation Layer still cannot directly access the SQL Database Container in the Data Layer.
4. **Auditability:** Regulators can audit specific layers (like the Data Layer's Audit Container) without accessing the entire system.
5. **Alignment with Partner Needs:** Banks prefer modular systems where each part can be certified and audited independently."

B. For Integration Mechanism with Partners:

- **Options:** Direct APIs, Central Middleware, Blockchain.
- **Choice:** Central Middleware (implemented as a dedicated Orchestration Layer with its own containers). This provides a single, auditable point of control for the Central Bank and simplifies connections for commercial banks.

3.4 System Framework (Layers + Containers)

- **Objective:** Present the big picture of the system as a set of logical layers, each containing a group of independent containers.
- **What to write:** Provide an illustrative diagram showing the layers and the containers within them, and the relationships (data flow) between containers across layers. Then explain the diagram in detail in the text.
- **Example (Customized for the Financial Project with Integrated Architecture):**
 "Figure (3.1) illustrates the proposed framework for the intelligent financial system, which integrates the concepts of **Layers** and **Containers**. The system is vertically divided into 8 logical layers based on function. Each layer contains several horizontal containers representing independent execution units.

Layer 1: Presentation Layer

- **Function:** Direct interaction with end-users and partners.
- **Containers in this Layer:**
 - **Mobile App Container:** User interface for iOS and Android.
 - **Web App Container:** User interface for web browsers (React/Angular).
 - **Partner Dashboard Container:** Specialized interface for the Central Bank and commercial banks to monitor transactions and generate reports.

Layer 2: API Gateway Layer

- **Function:** The single entry point for all requests; enforces security, authentication, and routing.

- **Containers in this Layer:**
 - **Auth Container:** Verifies user identity and issues JWTs.
 - **Routing Container:** Routes requests to the appropriate containers in lower layers.
 - **Rate Limiter Container:** Prevents abuse by limiting the number of requests per user/IP.
 - **Request/Response Transformer Container:** Adapts requests/responses to formats expected by internal/external systems.

Layer 3: Orchestration Layer (Workflow Management)

- **Function:** Manages complex, multi-step business processes that span multiple containers and layers.
- **Containers in this Layer:**
 - **Transfer Workflow Container:** Coordinates the steps for a transfer (Fraud Detection -> Price Prediction -> Accounting -> Bank Integration).
 - **KYC Workflow Container:** Coordinates identity verification steps (document upload, verification with external agencies).
 - **Settlement Workflow Container:** Manages end-of-day/netting processes with partner banks.

Layer 4: Business Logic Layer (Core Domain)

- **Function:** Executes the core accounting and business rules.
- **Containers in this Layer:**
 - **User Container:** Manages user profiles, KYC status, and authentication data.
 - **Wallet/Ledger Container:** Manages user balances across different currencies; the source of truth for funds.
 - **Transaction Processing Container:** Executes the core logic for transfers (debit/credit, fee calculation, validation).
 - **Market Data Container:** Fetches and serves live and historical exchange rates, fees, and market news.

Layer 5: Artificial Intelligence Layer (Intelligent Services)

- **Function:** Provides advanced analytical and predictive capabilities.
- **Containers in this Layer:**
 - **Forex Prediction Container:** Contains the trained LSTM model for forecasting exchange rates and providing recommendations.
 - **Fraud Detection Container:** Contains the trained XGBoost model for real-time transaction analysis and risk scoring.
 - **Risk Analysis Container:** Calculates overall risk profiles for users and transactions.
 - **Recommendation Container:** Suggests optimal transfer times, currencies, or products to users.

Layer 6: External Integration Layer (Partner Adapters)

- **Function:** Manages all communication with external partner systems.
- **Containers in this Layer:**
 - **Central Bank Integration Container:** Handles reporting, policy fetching (e.g., daily limits), and compliance checks.
 - **Commercial Banks Integration Container:** Contains adapters for each partner bank, translating unified internal requests into bank-specific API calls.
 - **Forex Providers Container:** Fetches live rates from multiple providers (Bloomberg, Reuters, local exchanges).
 - **Payment Gateways Container:** Integrates with card networks (Visa/Mastercard) for card-to-account funding.

Layer 7: Data Layer (Persistence)

- **Function:** Stores and manages all system data.
- **Containers in this Layer:**
 - **SQL Database Container (e.g., PostgreSQL):** For structured, relational data (users, wallets, transaction records).
 - **NoSQL Database Container (e.g., MongoDB):** For semi-structured data (market data, session logs).
 - **Cache Container (e.g., Redis):** For high-speed access to live rates, user sessions, and temporary data.

- **Audit Log Container (e.g., Elasticsearch):** For immutable, searchable storage of all transactions for regulatory audit.
- **Object Storage Container (e.g., MinIO):** For storing KYC documents and proof of transaction files.

Layer 8: Infrastructure & Observability Layer (Cross-Cutting)

- **Function:** Supports and monitors all other layers and containers.
- **Containers in this Layer:**
 - **Monitoring Container (e.g., Prometheus/Grafana):** Collects and visualizes metrics (CPU, memory, request rates) from all containers.
 - **Logging Container (e.g., ELK Stack):** Aggregates and analyzes logs from all containers.
 - **Alerting Container (e.g., Alertmanager):** Sends notifications based on defined rules (e.g., high error rate, container down).
 - **Service Discovery Container (e.g., Consul/Kubernetes DNS):** Allows containers to find each other dynamically."

Workflow Example (User Transfer Request):

1. **Layer 1 (Presentation):** The user enters transfer details in the **Mobile App Container**.
2. **Layer 2 (API Gateway):** The request hits the **Auth Container** (validates JWT) and then the **Routing Container**, which forwards it.
3. **Layer 3 (Orchestration):** The **Transfer Workflow Container** receives the request and orchestrates the process:
 - Calls **Fraud Detection Container** (Layer 5) for risk scoring.
 - Calls **Forex Prediction Container** (Layer 5) for the best rate.
4. **Layer 4 (Business Logic):** Upon approval, the workflow container calls:
 - **Wallet/Ledger Container** (Layer 4) to check and reserve balance.
 - **Transaction Processing Container** (Layer 4) to execute the accounting logic.
5. **Layer 6 (External Integration):** The workflow container calls the **Commercial Banks Integration Container** (Layer 6) to instruct the partner banks to move the funds externally. It also notifies the **Central Bank Integration Container** if required.
6. **Layer 7 (Data):** All steps are recorded by the **SQL Container** (balance update), the **Audit Log Container** (immutable record), and the **Cache Container** (temporary status).
7. **Layer 8 (Observability):** Throughout the process, the **Monitoring Container** tracks performance, and the **Logging Container** records every event.
8. A success/failure response is sent back to the user through the layers in reverse order."

3.5 Dataset Description

- **Objective:** To accurately document the data sources used for training the AI models and testing the system, ensuring research transparency and reproducibility.
- **What to write:** Detail the data type, source, description, size, and any preprocessing performed.
- **Examples (Customized for the Financial Project):**

A. Exchange Rate Data (for the Forex Prediction Container)

"The **"Forex Historical Data"** dataset from **Quandl** (now part of Nasdaq) was used. It is a globally recognized and reliable source for financial market data.

- **Data Type:** Time series data for major currency pairs (EUR/USD, GBP/USD, USD/EGP, etc.).
- **Source:** Quandl platform via API [Add link], aggregating data from global central banks and financial institutions.
- **Data Description:** Includes daily Open, High, Low, Close (OHLC) prices, Trading Volume, and Bid/Ask Spread. Data covers the period from 2010 to 2024.
- **Data Size:** Approximately 500,000 records, totaling ~1.5 GB after compression.
- **Modifications Made:**
 - Data from multiple files was consolidated into a single database.
 - Technical indicators (Moving Averages, RSI, MACD) were calculated and added as features.
 - Missing values were handled using forward-fill and interpolation methods suitable for time series.
 - Data was normalized using StandardScaler for the LSTM model."

B. Simulated Transaction Data (for the Fraud Detection Container & Testing)

"Due to the sensitive and private nature of real banking transaction data, a synthetic dataset was generated in collaboration with banking domain experts to realistically reflect real-world transaction characteristics.

- **Data Type:** Transaction log records.
- **Source:** Generated using Python's Faker library, with anomalies injected at a rate of 2% to simulate fraudulent activities based on known fraud patterns.
- **Data Description:** 1,000,000 simulated financial transactions over 5 years. Each record includes: Timestamp, User ID (anonymized), Source Currency, Target Currency, Amount, Applied Exchange Rate, User Location (Country), Transaction Status (Success/Failed/Suspicious).
- **Data Size:** Split into 70% training, 15% validation, and 15% testing sets.
- **Modifications Made:**
 - All personally identifiable information was anonymized.
 - Synthetic fraud cases were engineered with specific patterns (unusually high amounts, cross-border transactions to high-risk countries, odd-hour transactions).
 - Timestamps were converted into cyclical features (hour of day, day of week) to capture temporal patterns.
 - SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training data to address the class imbalance between normal and fraudulent transactions."

C. Central Bank Policy Data (for the Central Bank Integration Container)

"The "Central Bank Policy Rates" database from the Bank for International Settlements (BIS) was used.

- **Data Type:** Key interest rates, reserve requirements, and policy announcements.
- **Source:** Official BIS website [Add link].
- **Data Description:** Monthly updates of monetary policies for G20 countries from 2010 to 2024.
- **Modifications Made:**
 - Textual policy descriptions were converted into quantitative sentiment scores using a simple NLP model.
 - Data was aligned and merged with the exchange rate dataset to analyze policy impact."

3.6 Implementation Details (Per Layer/Container)

- **Objective:** Explain how each key container within its respective layer was built, specifying the technologies used.
- **What to write:** For the main containers, describe the base image, key libraries, the model/ logic inside, and the API it exposes.
- **Example (Customized for the Financial Project with Containers):**

"The entire system was containerized using **Docker**. Container orchestration, deployment, and auto-scaling are managed by a **Kubernetes** cluster.

A. Presentation Layer Containers:

- **Mobile App Container:** Built using **React Native**. The container itself is the compiled app package, served statically or via a CDN.
- **Web App Container:** Built using **React.js** with **Vite**. Served via an **Nginx** container.

B. API Gateway Layer Containers:

- **All containers in this layer** are based on the **Kong API Gateway** Docker image, configured via declarative YAML files.
- **Auth Container:** A custom plugin written in **Lua** that validates JWT tokens against the User Container's API.

C. AI Layer Containers:

- **Forex Prediction Container:**
 - **Base Image:** python:3.11-slim
 - **Libraries:** TensorFlow 2.15, Pandas, NumPy, Flask.
 - **Content:** Contains the pre-trained LSTM model (lstm_forex_model.h5) and a Python Flask app.
 - **API:** Exposes a REST endpoint /predict that accepts the last 60 days of features (JSON) and returns the predicted prices for the next 5 days (JSON).
- **Fraud Detection Container:**
 - **Base Image:** python:3.11-slim
 - **Libraries:** XGBoost, Scikit-learn, Pandas, Flask.
 - **Content:** Contains the pre-trained XGBoost model (xgb_fraud_model.pkl) and a Python Flask app.
 - **API:** Exposes a REST endpoint /analyze that accepts transaction details (JSON) and returns a risk_score (0-100) and a fraud_probability.

D. Business Logic Layer Containers:

- **Wallet/Ledger Container:**
 - **Base Image:** eclipse-temurin:17-jre-alpine (Java).
 - **Framework:** Spring Boot.
 - **Logic:** Implements services for checking balance, reserving funds, debiting/crediting wallets. Uses ACID transactions.
 - **Database:** Connects to the PostgreSQL Container in the Data Layer.
- **Transaction Processing Container:**
 - **Base Image:** node:20-alpine (Chosen for high I/O performance).
 - **Framework:** Express.js.
 - **Logic:** Contains the core logic for processing a transfer, calling the Wallet Container and the Ledger, and calculating fees.

E. External Integration Layer Containers:

- **Commercial Banks Integration Container:**
 - **Base Image:** eclipse-temurin:17-jre-alpine.
 - **Framework:** Spring Boot with WebClient for reactive calls.
 - **Content:** Contains adapter modules for each partner bank. Each adapter handles the specific authentication, request/response format (XML, JSON, ISO 8583), and communication protocol (HTTP, SOAP, MQ) required by that bank.

F. Data Layer Containers:

- All are standard, officially supported Docker images configured with necessary volumes and environment variables:
 - **SQL Container:** postgres:15-alpine
 - **Cache Container:** redis:7-alpine
 - **Audit Log Container:** elasticsearch:8 with logstash and kibana (the ELK stack).
 - **Object Storage Container:** minio/minio

G. Infrastructure Layer Containers:

- **Monitoring Container:** Runs prom/prometheus and grafana/grafana.
- **Logging Container:** Runs elastic/filebeat on each node to ship logs to the Audit Log's Elasticsearch."

3.7 Evaluation Plan

- **Objective:** Detail how you will prove that the entire integrated system, its layers, and its individual containers work correctly and meet the specified requirements.
- **What to write:** Describe the testing strategy at different levels (unit, integration, performance, security) and the specific metrics used.
- **Example (Customized for the Financial Project with Integrated Architecture):**

"The system was evaluated using a multi-level strategy, testing components from the smallest unit to the fully integrated system.

A. Container-Level Evaluation (Unit Testing / Component Testing):

- **Objective:** Verify the functionality of each container in isolation.
- **Method:** For each container, a suite of unit and integration tests was written.
- **Example:** The Fraud Detection Container was tested by sending 1,000 known transaction records (with labels) to its API and calculating **Precision, Recall, and F1-Score**.
- **Tools:** Pytest (Python), JUnit (Java), Jest (Node.js).

B. Integration Testing (Inter-Container & Inter-Layer):

- **Objective:** Verify that containers communicate correctly within and across layers, and that end-to-end workflows execute as designed.
- **Method:** Test complete user journeys, such as "Successful Fund Transfer" or "Failed Transfer Due to Fraud Flag".
- **Example:** A test would trigger the Transfer Workflow Container via the API Gateway and verify that it correctly calls the Fraud Detection, Wallet, Transaction, and Bank Integration containers in the correct sequence, and that the final state in the database is consistent.
- **Tools:** Postman / Newman, custom integration test suites.

C. Performance & Scalability Testing (System-Level):

- **Objective:** Ensure the system meets the non-functional requirements for response time and scalability under load.
- **Method:** Simulate realistic and peak load conditions.
- **Metrics:**

- **Average Response Time:** For a complete transfer request (target: < 3 seconds).
- **Throughput:** Transactions per second (TPS).
- **Container Auto-scaling:** Verify that Kubernetes automatically spins up new replicas of the Transaction Container under high load and scales them down when load decreases.

- **Tools:** Apache JMeter, K6.

D. Security Evaluation:

- **Objective:** Validate the system's resilience to attacks and compliance with security standards.
- **Method:**
 - **Container Image Scanning:** All Docker images were scanned using **Trivy** to identify and remediate vulnerabilities in base images and dependencies.
 - **Network Security Testing:** Verified that containers in the Data Layer are not accessible from the public internet and that communication between layers is encrypted (mTLS).
 - **Penetration Testing:** An external team conducted a penetration test on the staging environment, focusing on API endpoints and authentication mechanisms.

E. Model Performance Metrics (for AI Layer Containers):

- **Forex Prediction Container:** Evaluated using **Mean Absolute Percentage Error (MAPE)** to measure forecast accuracy and **Directional Accuracy (DA)** to measure the correctness of predicted price direction (up/down).
- **Fraud Detection Container:** Evaluated using **Precision, Recall, F1-Score**, and **Area Under the ROC Curve (AUC)** due to the imbalanced nature of the data."

3.8 Chapter Summary

- **Objective:** Briefly summarize what was covered in the chapter and provide a smooth transition to the next chapter (Results and Discussion).
- **What to write:** A concise paragraph reiterating the key components of the methodology.
- **Example (Customized for the Financial Project):**
-

"This chapter presented the detailed methodology for designing and implementing the intelligent, multi-currency financial transfer system. The core of the proposed approach is an **integrated architecture that combines logical layers (Presentation, API, Orchestration, Business, AI, Integration, Data, Infrastructure) with the technical flexibility of independent containers**. The chapter detailed the system's functional and non-functional requirements, justified the choice of this integrated architecture over alternatives, and provided a comprehensive framework diagram illustrating the 8 layers and their constituent containers. It also described the datasets used (Quandl forex data, simulated transactions, BIS policy data) for training the AI models housed in their respective containers. Finally, it outlined the implementation specifics for key containers and the multi-level evaluation plan (container, integration, performance, security) designed to validate that the system meets all requirements. The next chapter will present and discuss the results obtained from applying this methodology, including the performance of the AI models and the overall system under test."